MOSIP Decode 2026 — Problem Statement 1

Automated Conformance Testing for Inji Certify & Inji Verify

Findings Log and Open Questions

Shardul Chogale Mukta Varak

Document started: 20 September 2026
Last updated: 26 October 2026

Abstract

This document records, day by day, the conformance testing work carried out against the Inji stack as part of MOSIP Decode Problem Statement 1. It captures what was run, what failed, the evidence for each failure, and the questions we need answered by MOSIP mentors. Every finding is tagged as **CONFIRMED** or **UNVERIFIED**; we do not report a defect upstream until it is reproducible and the root cause is understood.

Contents

1 Scope and System Under Test **3**

2 Day Log **4**
2.1 Day 1 — Saturday, 20 September 2026 . . . 4
2.2 Day 2 — Tuesday, 23 September 2026 . . . 5
2.3 Day 3 — Wednesday, 24 September 2026 . . . 5
2.4 Day 4 — Monday, 29 September 2026 . . . 7
2.5 Day 5 — Wednesday, 30 September 2026 . . . 9
2.6 Day 6 — Friday, 2 October 2026 . . . 9
2.7 Day 7 — Monday, 26 October 2026 . . . 13
2.8 Day 8 — Tuesday, 27 October 2026 . . . 15

3 Findings Catalogue **17**
F-01 — Nonce has insufficient entropy . . . 17
F-02 — Nonce contains non-URL-safe characters . . . 18
F-03 — No DCQL support; test plan cannot proceed . . . 18
F-04 — Uses superseded `presentation_definition` . . . 18
F-05 — `client_metadata` absent in inline mode . . . 19
F-06 — No `direct_post.jwt`; HAIP certification impossible . . . 19
F-07 — DID document verification method identifier inconsistency . . . 19
F-08 — No mdoc / ISO 18013-5 support . . . 20
F-09 — Client identifier prefix has no overlap with 1.0 Final . . . 20
F-10 — Docker Compose database script out of date; every VP submission fails with HTTP 500 . . . 20
F-11 — Shipped DCQL sample uses the superseded format identifier . . . 21
F-12 — Certify's authorisation-server metadata cannot carry authorization_endpoint . . . 21
F-13 — Credential request rejects unrecognised parameters . . . 21
F-14 — Wrong error code for an unknown credential configuration . . . 22
F-15 — Authenticated wallets' proofs rejected in the pre-authorized flow . . . 22
F-16 — Revoked SD-JWT credentials are accepted: the status check silently does nothing 22

4 Open Questions for Mentors **24**

5 Reproduction **26**

1 Scope and System Under Test

Problem statement

Build an automated harness that spins up Inji Certify and Inji Verify, drives the OpenID Foundation conformance suite's REST API programmatically to run the issuer and verifier test plans, and feeds the results into each module's existing `api-testrig`, producing benchmark-gated reports with CI/CD integration.

Components under test

Component	Version	Notes
Inji Verify (service)	0.18.2	`injistack/inji-verify-service`
Inji Verify (UI)	0.18.2	`injistack/inji-verify-ui`, amd64 under emulation on arm64
Inji Certify	0.14.0	Not yet exercised (deferred, see Plan)
OpenID Conformance Suite	5.3.1 (rev 440eec8)	Local Docker deployment, prebuilt images

Test environment

- Host: Apple Silicon (arm64) macOS, Docker Desktop.
- Conformance suite reachable at `https://localhost.emobix.co.uk:8443` (self-signed certificate; hostname resolves publicly to 127.0.0.1).
- Inji Verify exposed publicly via an ngrok HTTPS tunnel. A public hostname is mandatory because Verify identifies itself with a `did:web` DID, which is resolved over public HTTPS.
- The conformance suite's REST API required no authentication token in the local `dev` profile, consistent with the problem statement.

Evidence handling

Conformance suite logs are exported as signed JSON (`.json` plus `.json.sig`) and stored under `logs/<date>/`. Findings below cite the exact log line (`src` field) and the OID4VP specification section reported by the suite.

2 Day Log

2.1 Day 1 — Saturday, 20 September 2026

Objective

Stand up the conformance suite and one MOSIP component, and complete one manual conformance test end to end to establish a baseline.

Decisions taken

1. **Inji Verify before Inji Certify.** Verify's Docker Compose brings up three containers with no external dependencies. Certify's `docker-compose-injistack` requires a PKCS12 keystore obtained through an onboarding process, a public DID endpoint, an external authorisation server, and partner API keys. The verifier test plan also has 12 modules against the issuer plan's 61. Verify is the cheaper path to a first result, and everything learned transfers.
2. **Non-HAIP plan first.** The HAIP verifier plan offers only `direct_post.jwt` as a response mode. Inji Verify hardcodes `direct_post` (see F-06), so no HAIP test can complete. We ran *OpenID for Verifiable Presentations 1.0 Final: Test a verifier — alpha tests* instead, which offers plain `direct_post`.
3. **Credential format `sd_jwt_vc`.** The plan offers only `sd_jwt_vc` and `iso_mdl`. Verify has no mdoc support whatsoever (F-08), making `iso_mdl` unusable.

Test plan configuration

- Plan: `oid4vp-1final-verifier-test-plan`, plan ID `lN7C4DH1HvYmc`
- Variant: `credential_format=sd_jwt_vc`, `client_id_prefix=redirect_uri`, `request_method=url_query`, `vp_profile=plain_vp`, `response_mode=direct_post`
- Alias: `injiverify-shardul`

Runs executed

Test ID	Outcome	Notes
`dgeeosZpIfDK1bA`	FAILED / INTERRUPTED	First attempt. Request object had expired (5-minute window). Discarded as an artefact of manual timing, not a defect.
`dMg4B5whxWgShFB`	FAILED / INTERRUPTED	Fresh request (2m05s old). Suite never dereferenced `request_uri` because the `url_query` variant expects inline parameters. Root cause understood; see F-09.
`MtzpWoD2dVgtic0`	FAILED / INTERRUPTED	After switching the credential's `clientIdScheme` to `pre_registered`. Reached 20+ checks; produced the substantive findings below. **This is the Day 1 baseline run.**

Configuration change made

In `inji-verify/docker-compose/config/config.json`, the credential *Mock Identity (SD JWT)* was changed from `"clientIdScheme":"did"` to `"clientIdScheme":"pre_registered"`. Per Inji Verify's own README, `did` delivers the authorisation request *by reference* via `request_uri`, whereas `pre_registered` delivers it *inline*. Only the inline form matches the `request_method=url_query` variant. Backup retained at `config.json.bak`.

Result of the baseline run (`MtzpWoD2dVgtic0`)

Approximately fourteen checks passed, including nonce extraction, response type, response mode, absence of `client_id_scheme`, absence of `scope`, absence of `redirect_uri`, response URI validity, and the outer parameter allow-list. Three checks failed and three raised warnings. The test terminated at `ExtractDCQLQueryFromAuthorizationRequest`.

Status at end of Day 1

Test 01 (`happy-flow`) complete and documented. Tests 02–11 deferred.

2.2 Day 2 — Tuesday, 23 September 2026

Calendar note

The work recorded above took place on 20 September, *three days before MOSIP Decode officially began.* The event kicked off on 23 September; the planned kickoff seminar and mentor AMA session did not take place. No work was carried out between 20 and 23 September, so everything listed as "deferred to Day 2" above remained outstanding at the start of the event proper. This is recorded because the project plan was originally dated from 20 September and therefore measured against the wrong calendar; it has since been re-anchored in `docs/PLAN.md`.

The practical consequence is favourable: the environment and a first evidenced baseline existed on day two of a four-week event, and the questions in Section 4 were already formulated before any mentor contact was possible.

Objective

Planning and documentation only. No conformance runs were executed, so no findings are added or amended in this entry.

Decisions taken

1. **Deliverable code location.** The conformance runner, plan configurations and orchestration live in our own repository. The `OpenIDConformanceTest` classes and their wiring go on branches of our forks of `inji-verify` and `inji-certify`, shaped so that each could be opened as an upstream pull request. Note that `api-test/` lives inside those two repositories, not in `mosip-functional-tests`, which carries only `apitest-commons`.
2. **Target specification treated as configuration.** Because Q1 has no channel for an answer other than the community forum, and may not receive one, the runner will take the plan name and variant set as configuration rather than hard-coding them. Either answer to Q1 then costs a configuration change rather than a rewrite.
3. **Benchmark semantics fixed provisionally as "regression from recorded baseline"** (option (a) in Q4), with an expected-failures file under version control. Given that the current honest baseline is a full set of failures at DCQL extraction, an absolute pass-rate gate would be meaningless.

Status at end of Day 2

No change to the findings catalogue. Modules 02–11, the ID2 verifier plan, and the resolution of F-05 remain outstanding and are scheduled within Week 1.

2.3 Day 3 — Wednesday, 24 September 2026

Objective

Build the Week 2 conformance runner, and before writing any of it, settle the two unknowns that `docs/WEEK2.md` identified as determining the week's shape. Both were resolved by reading the

conformance suite's source in the local clone rather than by experiment.

Unknown 1 resolved — the verifier handoff needs no browser

The verifier plans are inverted relative to the issuer plans: the suite plays the wallet and waits passively for the verifier to initiate. Its web UI presents this as a box into which a tester pastes an `openid4vp://` URI, which is why every run so far has been manual. The open question was whether that paste has an API equivalent, because if it did not, the entire week would have had to be re-planned around a semi-automated flow.

It does. `AbstractVP1FinalVerifierTest.start()` reads:

```
getBrowser().requestUriInput(env.getString("authorization_endpoint"),
"Paste the openid4vp:// authorization request produced by the verifier under
test; its query string will be delivered to this test's authorization end-
point.");
```

and `handleHttp()` dispatches on `path.equals("authorize")`. The test exposes `authorization_endpoint` via `exposeEnvString`, readable through `GET /api/runner/{id}`. Delivering an authorisation request is therefore an ordinary HTTP GET to that endpoint carrying the verifier's query parameters. The paste box is a convenience wrapper over exactly that call.

Consequence. A fully unattended verifier run is achievable. This substantially answers Q6, which we had expected to need a mentor.

Unknown 2 resolved — the screenshot gate is configuration, not manual work

The happy-flow module requires a screenshot of the verifier's verification-result page before reaching REVIEW. `AbstractVP1FinalVerifierTest` carries a purpose-built path for automated runs: `handleVerificationEvidenceRequest()` serves an HTML stand-in page, and `fillScreenshotPlaceholderViaBrowserAutomationIfConfigured()` fills the placeholder when the test configuration contains a `browser` automation entry matching the evidence URL. The source comment is explicit that this exists because "in automated runs there is no verifier UI a human could take a real screenshot of".

A `browser` block has been added to `configs/runner.json` accordingly. **This is unconfirmed against a live run** — the match pattern in particular is a reasonable guess at the evidence URL and must be validated before it is relied upon.

Work completed

- `runner/` package: configuration loading, a conformance suite API client, an Inji Verify client, the handoff, result normalisation, the benchmark gate, an orchestrator and a CLI entry point.
- `run-conformance.sh` providing `-component verify | -component certify | -combined`, as the problem statement requires.
- 24 unit tests, all passing, covering normalisation against the real Day 1 log and the gate's behaviour — including its failure path, since a gate that cannot fail is decoration.

Defect found in our own code

The gate initially set `regression` and `improvement` only on the branches that made them true, relying on the normalisation layer having pre-populated the defaults. Applied to any module assembled by another path this raised `KeyError` rather than reporting "no regression". Caught by `test_gate.py` before any live run and fixed by setting both unconditionally. Recorded because it is the argument for writing the gate's failure-path tests first: the bug lived in the success path and only the absent-key case exposed it.

Status at end of Day 3

Runner complete and unit-tested, but **not yet executed against a live conformance suite** — Docker was not running on the development machine during this session. Nothing in this entry should be read as evidence that the harness works end to end. The first live run is the next task, and until it happens the handoff and screenshot mechanisms are source-derived expectations rather than confirmed behaviour.

No new findings against Inji Verify; no conformance modules were executed.

2.4 Day 4 — Monday, 29 September 2026

Objective

First execution of the harness against a live conformance suite. The Day 3 runner had been unit-tested only, and every claim about the handoff was source-derived. Today's question was whether the automated run measures the same thing as the manual Day 1 run — not merely whether it completes.

Environment

Conformance suite 5.3.1 (prebuilt images), Inji Verify 0.18.2, ngrok 3.39.11 on the static domain. Both uncommitted local modifications (the ngrok host in `docker-compose.yml`, `pre_registered` for *Mock Identity (SD JWT)*) confirmed intact after five days. Same variant as Day 1.

Runs executed

Test ID	Outcome	Notes
(none)	HARNESS ERROR	Run 1. Suite side worked; Inji Verify returned HTTP 400 to `/vp-session-request`. Our defect, not Verify's — see below. No module result.
`hcuWEvxGbvRgt7e`	FAILED / INTERRUPTED	Run 2, `nonceMode=sdk`. **First fully unattended run.** Reproduces the Day 1 manual baseline exactly.
`Q9MaDZyfpJhjiDp`	FAILED / INTERRUPTED	Run 3, `nonceMode=service`. Controlled experiment; differs from run 2 in the nonce source only.

Result 1 — the automated run reproduces the manual baseline exactly

Run 2 and the Day 1 manual baseline (`MtzpWoD2dVgtic0`) produced identical counts — 15 success, 4 failure, 3 warning, 7 info — and the identical set of seven non-success checks, terminating at the same place, `ExtractDCQLQueryFromAuthorizationRequest`. No check appears in one run and not the other. The handoff delivered with the full 300 seconds remaining, and the suite answered its `/authorize` endpoint with HTTP 302.

This is the result that validates the harness. Completing is not enough; a harness that completed while measuring something different from what a tester would observe by hand would be worse than none. The Day 3 expectation that delivery is an ordinary HTTP GET to the exposed `authorization_endpoint` is now confirmed behaviour, not inference.

Result 2 — controlled experiment confirms the F-01/F-02 root cause

On Day 2 the weak nonce was traced to `btoa(Date.now())` in the SDK, with `verify-service`'s own `SecureRandom` generator reached only as a fallback. That was a reading of the source. Runs 2 and 3 test it: they differ in one variable — whether the harness supplies the SDK-shaped nonce or lets `verify-service` generate one.

Check	sdk (run 2)	service (run 3)
VP1FinalEnsureMinimumNonceEntropy	WARNING, 71.68 bits	SUCCESS
CheckForInvalidCharsInNonce	FAILURE (=)	SUCCESS
Nonce sent	MTc5MDY5NjY0NTQzNw==	78cb380cda818ebc189b80704e6f17a3
Counts (success / failure / warning)	15 / 4 / 3	17 / 3 / 2

Exactly the two nonce checks move; every other check is unchanged and both runs still terminate at DCQL extraction. F-01 and F-02 are therefore confirmed by three independent routes: the suite flags them, the source locates them in the SDK, and removing only the SDK's nonce clears them. The service-generated nonce is 32 hexadecimal characters — the 16 `SecureRandom` bytes of `SecurityUtils.generateNonce()`, as the source predicted. This materially strengthens the upstream report.

Measurement decision recorded: the nonce source

Driving `verify-service` directly rather than through the UI changes what is measured. If the harness sends no nonce, the service generates a sound one and the nonce findings silently vanish — not because anything was fixed, but because the harness bypassed the component that causes them. The runner therefore makes the nonce source explicit (`nonceMode`), defaults to `sdk` so that runs measure what the shipped product sends a wallet, and records the mode on every module's output. The SDK-shaped nonce is a deliberate, documented reproduction of the SDK line, verified against the three Day 1 nonces by a unit test.

Defects found in our own code

Four, all fixed today. Recorded because each would have produced wrong evidence.

1. **The harness sent no presentation definition.** `VPRequestCreateDto` takes it inline. Found by reading the DTO before the first run.
2. **The harness sent the raw config definition, which has no `id`.** `VPDefinitionResponseDto` requires one, so `verify-service` answered with a bare 400 (run 1). The UI wraps selected credentials in an envelope in `vpVerificationState.ts` (lines 58–63, 82–83): a fixed id, a fixed purpose, and *only* the credentials' `input_descriptors`. The harness now builds the same envelope.
3. **The gate passed a run that measured nothing.** Run 1's harness error had no result; the severity ordering read that as FAILED, equal to the FAILED baseline, and the gate reported PASSED. A harness error now always fails the gate while the module itself stays SKIP, so the fault is never recorded against MOSIP. *This is the most serious of the four*: with the current baseline, the harness could have broken completely and CI would have stayed green.
4. **A refactor deleted an exception class**, breaking the CLI while all unit tests passed — none imported the modules that used it. An import test now covers every runner module.

Observation — not a finding

The presentation definition id is a hard-coded constant in `vpVerificationState.ts:59`: every request from every deployment of this UI carries `c4822b58-7fb4-454e-b827-f8758fe27f9a`. Presentation Exchange describes the id as identifying the definition, and a constant is arguably fine for an unchanging definition — but here the definition's contents vary with which credentials are selected while the id does not. Not catalogued: we have not established that anything relies on the id being unique, and without a consequence it is not a defect. Raised as a question instead.

Status at end of Day 4

The harness runs unattended end to end and reproduces the manual baseline check-for-check. Carried-forward Week 1 items remaining: modules 02–11 and the ID2 verifier plan (now a single command each), and F-05, which today's runs cannot settle — the harness forwards `client_metadata` only when Verify supplies it, and in the `pre_registered` path it does not, so the open question of whether the UI adds it at QR-build time still needs the UI's QR payload. The screenshot mechanism is not yet exercised: both runs terminated at DCQL before reaching it.

Automated logs are archived unsigned under `logs/2026-09-29/`; anything reported upstream still needs the suite's signed export.

2.5 Day 5 — Wednesday, 30 September 2026

Objective

Prepare for the first mentor contact, and turn the project's documentation into a single starting point.

Facts established

- **The final submission deadline is 26 October 2026, 11:59 PM IST**, read from the official event page. Every plan so far assumed about 21 October; the real date adds five days, now used as a fifth week for finishing and submitting.
- **Mentor contact is the Weekly Connect**: Wednesdays, 5:00–6:00 PM IST, from today (from the organisers' email of 22 September). Planned checkpoints follow it: 30 September, 7, 14 and 21 October.
- Submission requires an overview plus a 1–2 page document or a 5–7 slide pitch; multiple submissions are allowed and the last before the deadline counts. The detailed submission and evaluation criteria sit behind separate links and are to be confirmed at the session.

Work completed

`docs/handbook/handbook.tex`: a mentor brief ranking the open questions by how much each answer changes the build, with our default for each; a start-to-end guide from a clean machine to shutdown and submission; a record of every step taken since 16 September; and a day-by-day planner for both of us to 26 October. `PLAN.md`, `CONTEXT.md` and `WEEK2.md` updated to the real deadline.

New question raised

The problem statement asks for Inji Certify and Inji Verify "on 1.0", with "1.0.x" repositories. The newest images we could run are Verify 0.18.2 and Certify 0.14.0. If "1.0" names a release line we are not running, every result so far is against the wrong version. Added to the mentor brief as the second-ranked question.

Status at end of Day 5

No conformance runs today; no findings added or changed. No reply yet from Mukta to the 29 September setup email. Mentor answers to be recorded here after the session.

2.6 Day 6 — Friday, 2 October 2026

Objective

Absorb the recording of the 23 September kickoff webinar, and act on anything in it that changes the build.

Correction to earlier entries

Earlier entries, and `PLAN.md`, said the kickoff seminar did not take place. **It did**: the organisers held it on 23 September, 5:00–6:00 PM IST, and recorded it. We did not attend live, and worked from the recording's transcript today. The "AMA" is the weekly Wednesday session, 5:00–6:00 PM IST, from 30 September; organisers' contacts are `decode@mosip.io` and the forum at community.mosip.io.

What the kickoff said about PS1

Walkthrough at 24:58–32:45 of the recording, plus the general session. Points that bear on our build:

- **Mandatory submissions** (08:01): source code, a working prototype, *a slide deck, and a video demo*. Stronger than the event page's "document *or* slides". Settles part of mentor-brief question 6: the video is required.
- **Docker Compose must itself support both modes** (28:30): one file or step that brings up each module on its own, or both together, connected to the conformance suite. This is a concrete design input for the compose work: selective bring-up, for example compose profiles, not one monolithic stack.
- **One-command execution is the core ask** (29:20–29:46): explicitly contrasted with configuring the plan, clicking continue and selecting tests in the suite's UI. The runner meets this for Verify.
- **CI/CD is framed as good-to-have, with a stated purpose** (27:33–28:26): run the suite on every feature change so non-conformance is caught when the change is made, not at the end. This is the case for our regression-from-baseline gate.
- **Result diff is motivated by run-to-run variation** (30:12–30:37): "each run might have different failures". Supports building the diff on module- and check-level results, which the contract already carries.
- The resources point to "the latest release 1.0.x" documentation (32:08). Taken with the problem statement's "on 1.0", this prompted the version check below.

Finding — an Inji 1.0 line exists, and most of our findings do not apply to it

The Interop Playground walkthrough (41:08) mentions choosing between a "presentation exchange request or a DCQL request" — implying some Inji Verify sends DCQL, which contradicts F-03. Checked against upstream rather than taken on trust:

- `mosip/inji-verify` has a `release-1.0.x` branch, under active development (head `6a6ab3b`, 1 October: "Rename DCQL error codes to snake case"), versions `1.0.0-alpha.2-SNAPSHOT`, and tags `v1.0.0-alpha.1` (4 August) and `v1.0.0-dev.*`. `mosip/inji-certify` also has `release-1.0.x`.
- Published on Docker Hub: `injistack/inji-verify-service` and `-ui` at `1.0.0-alpha.1`. The branch's compose file names `1.0.0-alpha.2`, which is not yet published.
- **Everything we have run was 0.18.2**, the newest image in the `0.18.x` line and the one the default branch's compose file pinned on 18 September.

Our findings against the 1.0 line, from source (not yet from runs):

Finding	0.18.2 (tested)	1.0.0-alpha.1 (runnable)	release-1.0.x (head)
F-01/F-02 nonce	`btoa(Date.now())`	`crypto.getRandomValues`	`crypto.getRandomValues`
F-03 DCQL	absent	present (16 service files)	present
F-06 response mode	`direct_post` only	`direct_post` only	not determined by source search

Finding	0.18.2 (tested)	1.0.0-alpha.1 (runnable)	release-1.0.x (head)
F-09 client id	`did`, `pre_registered`	`redirect_uri:` works — see runs below†	`x509_san_dns` configured

Consequences.

1. **The F-01/F-02 upstream issue is on hold.** As drafted it reports a defect the 1.0 line has already fixed. It may still be worth raising for the 0.18.x line if that line is maintained — a question for the mentors, not an assumption. This is the "confirm before claiming" rule doing its job a third time.
2. **The baseline should move to `1.0.0-alpha.1`.** It is the newest runnable 1.0 build and contains DCQL, so the OID4VP 1.0 Final plan may now produce real passes rather than a uniform failure at DCQL extraction. This would also give the gate something to protect, which it currently lacks.
3. **Every finding in the catalogue is now scoped to 0.18.2**, explicitly, below. They remain true and evidenced for that version.
4. The runner's `sdk` nonce mode reproduces the 0.18.x SDK. Against 1.0 builds the faithful reproduction is a random nonce; the mode needs to follow the version under test rather than be a fixed default.

† Corrected the same day. Written from source as "no overlap" because alpha.1 has no `x509_san_dns`; run `5KS13qvmbwOMNtY` below showed the `redirect_uri:` client identifier prefix works, which source search had not revealed. Recorded rather than silently fixed.

Runs on Inji Verify 1.0.0-alpha.1

Run from a worktree of `inji-verify` at tag `v1.0.0-alpha.1` (`../inji-verify-1.0`) with the same suite and variant as every earlier run. The runner gained a 1.0 mode first: `/v2/vp-session-request`, a DCQL query read from Verify's own config, and a reproduction of the 1.0 SDK, including the `client_metadata` it adds when building the request URI. Each run changed one thing, so each change in result has a single cause.

Test ID	Change from previous	Result	Stopped at / outcome
`Bob121tXCWTGf9y`	First 1.0 run; shipped *Mock Identity* credential	INTERRUPTED / FAILED, 23 pass	`CreateSdJwtKbCredential`: our plan config lacked a signing key. **DCQL extracted; all nonce checks pass.**
`gIA512EeJWehcHt`	Test signing key added	FINISHED / FAILED, 28 pass	Verify answered the submission with **HTTP 500** — F-10
`tpKVz6upccxpO5U`	Database init script corrected	FINISHED / FAILED, 28 pass	Verify answered 400: credential `vct` does not match the query — correct behaviour
`ZQqnbYfw46wpmyI`	Credential requesting `urn:eudi:pid:1` as `dc+sd-jwt`	FINISHED / FAILED, 32 pass	**Verify accepted the presentation (HTTP 200).** Two DCQL failures cleared. Remaining failures: client id pairing only

Test ID	Change from previous	Result	Stopped at / outcome
5KS13qvmbwOMNtY	Client id redirect_uri:<response_uri>	**FINISHED / REVIEW, 35 pass, 0 fail, 0 warn**	Every automated check passes; screenshot placeholder filled by the suite's browser automation

The happy-flow module passes against Inji Verify 1.0.0-alpha.1, fully unattended. REVIEW is the suite's state for "all checks passed, image evidence awaiting human review", which this module reaches by design. The suite log confirms "All placeholders filled by scripted browser" — the screenshot requirement (Q6) is automated, now observed rather than read from source.

What a passing run requires — each item established by the run that needed it:

1. A signing key for the suite's test credential (`credential.signing_jwk`), generated with the suite's own `generate-vp-test-cert.py`; kept out of git.
2. The database fix for F-10.
3. A verifier credential requesting `vct urn:eudi:pid:1` as `dc+sd-jwt`. The suite's verifier tests *always* present that credential type — hard-coded in `CreateSdJwtKbCredential.java:17` — so a verifier asking for anything else will, correctly, reject it. Run `tpKVz6upccxpO5U` is Inji Verify doing exactly that. Added locally to Verify's `config.json`.
4. The `redirect_uri:` client identifier prefix: `client_id` = `redirect_uri:` followed by the response URI, matching the plan's `client_id_prefix=redirect_uri` variant. Per-developer, because the response URI contains the ngrok host.

The gate now has something to protect. The 1.0 baseline records happy-flow as REVIEW; a return to FAILED on the target version fails the build — which, while every module failed, the gate could never do. Baselines are now per version (`expected-failures.json` for 1.0; `expected-failures-0.18.json` for the earlier evidence).

The full plan on Inji Verify 1.0.0-alpha.1

Plan `nupFFSGqySsbb`: all 11 modules the OID4VP 1.0 Final verifier plan contains for this variant (`invalid-session-transcript` is mdoc-only). Every module FINISHED with 0 failures and 0 warnings: 4 PASSED, 7 REVIEW.

REVIEW was not taken at face value. For the negative tests the suite presents a forged credential and asks a human to confirm the verifier rejected it. It cannot see this itself: Inji Verify answers the `direct_post` with HTTP 200 and verifies afterwards, which OID4VP 1.0 Final permits. Our screenshot automation fills the placeholder regardless — so "REVIEW, 0 failures" would have been reported even if Verify had *accepted* a forged credential. The first full run (plan `YyUQFGfA9rlMr`) had exactly that blind spot.

The harness now settles each REVIEW with evidence only the verifier side has: Inji Verify's own verdict from `/vp-session-results`, compared with what the suite's REVIEW message says the test requires. A contradiction fails the gate. Second full run:

Module	Test ID	Suite	Inji Verify's verdict
happy-flow	FiEKKnONeoNFVWu	REVIEW	accepted; required accept — consistent
minimal-cnf-jwk	eGNvPaFZCvXDyF2	REVIEW	accepted; required accept — consistent
request-uri-method-post	D9k8YD8bC5KOPYk	REVIEW	accepted — **not exercised** (n/a to `url_query`)
request-uri-fetched-twice	1m9l4wYovOH7Qq5	REVIEW	accepted — **not exercised** (n/a to `url_query`)
invalid-kb-jwt-signature	V8O4tr84SWq2nRX	REVIEW	rejected; required reject — consistent
invalid-credential-signature	R7r9vitOsOOMGiG	REVIEW	rejected; required reject — consistent
invalid-sd-hash	jP5ItCI3EM9AT2w	REVIEW	rejected; required reject — consistent
invalid-kb-jwt-nonce	79k2ff0JPhb1Db6	PASSED	rejected at submission (HTTP 400)
invalid-kb-jwt-aud	ACAbYUZ9qjIYl6E	PASSED	rejected at submission (HTTP 400)

Module	Test ID	Suite	Inji Verify's verdict
kb-jwt-iat-in-past	`wXtLgQRsyy2VjF2`	PASSED	rejected at submission (HTTP 400)
kb-jwt-iat-in-future	`LJnVhzjO72E8sEe`	PASSED	rejected at submission (HTTP 400)

Result. On Inji Verify 1.0.0-alpha.1 the verifier accepted every valid presentation and rejected every invalid one — three forged ones (credential signature, key-binding signature, SD hash) after accepting the submission, and four (nonce, audience, issued-at past and future) at submission with precise error messages. **Nine modules are genuinely exercised and all nine are consistent with conformance.** Two, request-uri-method-post and request-uri-fetched-twice, test fetching the request by reference; the suite's own descriptions say they do not apply to `request_method=url_query`, so under our variant they ran as plain happy flows. They are recorded as *not exercised*, not as passes. Exercising them needs a signed by-reference request with `x509_san_dns`, present only on the `release-1.0.x` head.

The 1.0 baseline now covers all 11 modules, each with its reason and evidence; re-gating this run against it passes with no unknown modules and no mismatches.

Status at end of Day 6

The verifier plan is complete on the target version: 9 of 11 modules exercised, all consistent with conformance; 2 not applicable to our variant. Earlier in the day, before the full run: one module of twelve passed. Two new findings (F-10, F-11); F-05 resolved as our artefact; F-01, F-02, F-03, F-04 confirmed fixed on 1.0 by run; F-07 persists on 1.0; F-09 does not apply on 1.0. Next: Inji Certify. Question for mentors, sharpened: is `1.0.0-alpha.1` the target, or the `release-1.0.x` head built from source?

2.7 Day 7 — Monday, 26 October 2026

Context

No work took place between 2 and 26 October. On 26 October the organisers extended the submission deadline to 2 November 2026, and an earlier notice (12 October) confirmed that submissions go by email to `decode@mosip.io`, not through the Unstop page. Mukta's assigned work had not started; the final week is re-planned with Shardul covering it.

Work completed — problem statement task (iii)

`OpenIDConformanceTest` added to Inji Verify's api-test, on branch `feature/openid-conformance-test` of `shard-c6/inji-verify`, based on `release-1.0.x`. It reads the runner's contract document — or invokes the runner against the testrig's own `injiVerifyBaseUrl`, via a new `-endpoint` option — and reports each conformance module as a TestNG test, plus one test for the benchmark gate.

Verified three ways, by running the class under TestNG with MOSIP's own report listener:

- the real full-plan run on Inji Verify 1.0.0-alpha.1: 12 of 12 tests pass, each named `conformance.inji-verify.<module>` in MOSIP's EmailableReport;
- a doctored copy with one regression and one accepted forged credential: exactly those two and the gate fail, each with a specific reason;
- nothing configured: one test, skipped, with an explanation — existing testrig runs are unaffected.

Environment note

Building api-test inside a git worktree needs `-Dmaven.gitcommitid.skip=true`: the project's `git-commit-id-plugin` cannot read a worktree's `.git` file. A property of our setup, not of Inji Verify; not a finding.

Docker Compose stack and CI

The suite and Inji Verify 1.0 now run from `docker/docker-compose.yml` in our repository, with per-module profiles, and with Inji Verify's two fixed files (F-10's database script, the conformance credential) kept in the repository instead of edited by hand. CI runs the harness tests and validates the stack on every push; the first run on GitHub passed.

The full verifier plan run on the Compose stack (plan `2WXPDhBNMwyCL`) is identical to the 2 October run module for module and check for check, and passes the gate against the full baseline. Fed to `OpenIDConformanceTest`, the same result gives 12 of 12 passing tests in MOSIP's EmailableReport: stack, runner, gate and api-testrig shown together on one run.

Inji Certify, time-boxed

Brought Certify 1.0 into the Compose stack (profile `certify`) and taught the runner the issuer side: it asks Certify for a pre-authorized offer — with a transaction code — and delivers it, then the code, to the suite. Both components share the one ngrok domain through a gateway. Certify's own signing keystore is generated at start-up: the "PKCS12 from Mimoto onboarding" blocker recorded since September applies to Mimoto, which conformance does not need.

Configuration we had to get right, each found by a run: Certify as its own authorisation server; its credential endpoint validating its own tokens (not eSignet's); an SD-JWT credential, since Certify ships only `ldp_vc`; the transaction code. Then the full issuer plan, 15 modules for the pre-authorized / SD-JWT / DPoP variant: 3 PASSED — each confirmed in Certify's log to be rejected for the right reason, so none is vacuous — 3 SKIPPED for features Certify does not offer, 1 WARNING, and 8 FAILED, every one traced to F-12–F-15 or an optional feature. All results through the F-12 shim. Baseline recorded for all 15.

Inji Certify's api-testrig and the combined run

`OpenIDConformanceTest` added to Inji Certify's api-test too (branch `feature/openid-conformance-test` of `shard-c6/inji-certify`, based on `release-1.0.x`): the same class as Inji Verify's, running the runner against the testrig's own `injiCertifyBaseURL` when asked to. Verified the same three ways: the recorded issuer run gives 16 of 16 passing tests (15 modules and the gate); a doctored copy with one regression fails exactly that module and the gate; with nothing configured, one skipped test.

Then the combined run, `./run-conformance.sh -combined` on the `combined` stack: both plans in one invocation, one result document, one gate. Inji Verify (plan `uzQR679oonrne`, 11 modules) and Inji Certify (plan `q4RAMcjJWpAhF`, 15 modules) each came out identical, module for module and check for check, to their standalone runs; the gate passes with no regressions, unknown modules, harness errors or review mismatches. That one file, fed to both api-testrig classes, gives 12 of 12 and 16 of 16 passing tests.

Closing the gaps against the problem statement

A requirement-by-requirement check against the problem statement's text (`docs/STATUS.md`) found four things short of what it asks, each now closed:

- **Task (iii), runner mode never run live.** Both classes had only read result files. Run in runner mode, the Certify class first handed the runner a relative endpoint: with TestNG started directly the testrig's configuration is never loaded and its base URL is empty. The class now fails setup with a clear message unless it has an absolute URL, and accepts the URL as a JVM property. Then, live: the Certify class ran the runner (plan `UByUkX1NUvsK6`) and reported 16 of 16; the Verify class (plan `OF4joC3VfcDh7`) 12 of 12. A property of our test setup, not of either component; not a finding.
- **Task (iv), no human-readable consolidated report.** `-report` now writes one report (HTML and Markdown) covering every component in the run, and a status badge.

- **Task (v) and deliverable (c), two commands, not one.** `run-full-stack-conformance.sh` brings the stack up, runs, reports and shuts down; run end to end in combined mode (plans `g590BHmwSppVo`, `sYMCFZ5C9a6RA`).
- **Deliverable (d), CI ran Verify only.** The workflow, renamed to the problem statement's `full-stack-conformance.yml`, now runs verify, certify or combined on demand, combined weekly, and puts the report in the run summary. Not yet run on GitHub: it needs the two repository secrets.

Documentation (deliverable (e)) written: `ARCHITECTURE.md` with the architecture diagram, `SELF-CERTIFICATION.md`, `TROUBLESHOOTING.md`. Checking the OpenID Foundation's certification pages showed that certification requires the HAIP test plans (OID4VP with `direct_post.jwt`; OID4VCI with the authorization code flow and client attestation), which neither component can run yet; the plans we run are the right ones for catching regressions but are not the certification run. Recorded in the guide.

Status at end of Day 7

All five mandatory tasks are demonstrated end to end: (i) one Compose stack for the suite and both components; (ii) the runner; (iii) `OpenIDConformanceTest` in both api-testrigs; (iv) the combined run; (v) one command each. GitHub Actions in place. No findings changed.

2.8 Day 8 — Tuesday, 27 October 2026

Fresh-clone test

A clean clone from GitHub, set up by following only the README. Three gaps found and fixed: the per-developer settings (ngrok domain, client id, the two sets of test keys) were described only in the handbook, now in the README with an example file to copy; `cryptography`, needed to generate the issuer plan's client keys, was missing from `requirements.txt`; a missing key file ended in a Python traceback, now a clear setup error (exit status 2). The clean clone, with newly generated keys, then ran the one-command combined run: gate passes, and the new result diff shows it identical to the 26 October combined run across all 26 modules.

Result diff (good-to-have iv)

`python -m runner.diff` compares two runs module by module — suite result, gate verdict, and which checks did not succeed (by condition, ignoring messages that change every run). It confirms what was checked by hand on Day 7: the combined run equals both standalone runs.

Signed exports

Exported the suite's signed zips for the evidence plans (one signature per module) into a gitignored `evidence/` folder for the submission. They are not committed: the suite deliberately leaves private keys in its logs, a signature prevents redacting them, and these contain our test keys. The keys are test-only and are regenerated after sharing.

First full run on GitHub Actions — and a moving suite

With the two repository secrets added, the full combined run ran on GitHub (run 6): stack up, public tunnel, both plans, gate passed, report attached. Its result was *not* identical to ours. CI pulled the suite's `latest` image, published that morning; our machine had the 19 September build. Every module the two builds share gave the same result, but the verifier plan changed: `request-uri-fetched-twice` and `request-uri-method-post` dropped, `present-revoked-credential`, `present-credential-without-status` and `wrong-vct` added; the issuer plan dropped `fail-unsupported-encryption-algorithm`. The gate passed only because modules without a baseline are reported, not failed.

Decision. Test against the latest suite, but pin it by digest (`docker/docker-compose.yml`): a baseline means nothing against a suite that can change overnight. Updating is documented in `TROUBLESHOOTING.md`. Locally, the pinned build (plans `dk7WTeEpVfDCW`, `9soA2jgYi4lki`) gave a result identical to CI run 6 across all 26 modules. Baseline moved to it, every new entry checked against its log; then plans `tsEPVoEXK87hL` and `kQsIvua57Vdib`: gate passes, no module without a baseline.

F-16 found. `present-revoked-credential` fails, and Verify's own verdict accepts the revoked credential. Traced through both code bases before recording it (catalogue, F-16).

Gate rule, narrowed. A verdict contradiction always failed the gate, so the target version would have stayed red until MOSIP fixes F-16 — and a gate that is always red gets ignored. The baseline may now acknowledge a contradiction, but only when the suite itself already failed the module and the entry names the finding. A contradiction the suite cannot see (a REVIEW, such as a forged credential accepted) still always fails. Four tests pin this; the known defect is printed on every run and at the top of the report.

Inji forks brought up to date

Both forks' `feature/openid-conformance-test` merged with upstream `release-1.0.x` (Verify 18 commits, Certify 11; no conflicts). Upstream renamed the next release `1.0.0-rc.1`. Both api-testrig classes rebuilt and ran the runner against the new stack: Verify 13 of 13 (plan `QpHhlq7hq6TLz`), Certify 15 of 15 (plan `pygcrOj5sEs4Q`). The images under test are unchanged: Certify's pinned QA build (12 October) is still the newest published; for Verify a QA build of `release-1.0.x` (21 October) is newer than `1.0.0-alpha.1` and is the next step, kept separate so a changed result can be attributed.

Status at end of Day 8

All mandatory tasks and all good-to-haves done; reproducible from a fresh clone; the full run proven on GitHub Actions. One new finding (F-16). Left: Verify's newer QA build, slide deck, video, contribution record, college IDs, submission email.

3 Findings Catalogue

Severity reflects impact on certifiability and on security, not on our convenience.

Version scope (added 2 October). Every finding below was established against **Inji Verify 0.18.2**. On 2 October we found an Inji 1.0 line (`release-1.0.x`; runnable as `1.0.0-alpha.1`) in which F-01, F-02 and F-03 appear fixed from source and F-09 is partly addressed. The findings stand for 0.18.2; none is to be reported upstream until it has been re-checked against the 1.0 line. See the Day 6 entry. F-10 and F-11 were found on the 1.0 line itself.

F-01 — Nonce has insufficient entropy HIGH CONFIRMED

Component: Inji Verify 0.18.2 **Spec:** OID4VP 1.0 Final, §5.2

Observation. The `nonce` in Verify's OID4VP authorisation request is a base64-encoded millisecond Unix timestamp rather than a cryptographically random value.

Evidence — our analysis. Three samples decoded, across both request-construction paths:

clientIdScheme	nonce (base64)	decoded	Δ to issuedAt
`did`	`MTc4OTg4MzIxMzQ5MA==`	1789883213490	—
`pre_registered`	`MTc4OTg4NDUzNjY3OQ==`	1789884536679	120 ms
`pre_registered`	`MTc4OTg4NDY1NjQzNg==`	1789884656436	158 ms

Evidence — conformance suite, independently.

> `VP1FinalEnsureMinimumNonceEntropy` — WARNING
> "Nonce does not appear to have sufficient entropy (calculated Shannon entropy is below the threshold). Section 5.2 of OID4VP 1.0 Final requires the nonce to be a 'fresh, cryptographically random number with sufficient entropy'; OpenID4VP PR #722 recommends at least 128 bits."
> `actual: 73.68 expected: 96.0`

Root cause — located 23 September, and it corrects our Day 1 attribution. The weak nonce is generated by the *SDK*, not by the service. In `inji-verify-sdk/src/utils/api.ts` lines 10–12:

```
const generateNonce = (): string => {
return btoa(Date.now().toString());
};
```

used at `api.ts:89` and `api.ts:159`. `verify-ui` depends on `@injistack/react-inji-verify-sdk` 0.18.1, so this is the nonce that reaches the authorisation request. `btoa` is standard base64, which pads with `=` — the same line therefore causes F-02.

The service already implements this correctly. `verify-service/.../utils/SecurityUtils.java` generates a nonce from `SecureRandom` over 16 bytes. But at `VerifiablePresentationRequestServiceImpl.java:86` it is only a fallback: `vpRequestCreate.getNonce() != null ? vpRequestCreate.getNonce() : SecurityUtils.generateNonce()`. Because the SDK always supplies a nonce, the sound generator never executes in the normal flow.

This reframes the finding materially: it is *not* a missing capability but a correct implementation overridden by its caller, which makes it a small, well-scoped fix rather than a design change — and a much more actionable report for a maintainer. Our Day 1 write-up attributed it to Inji Verify generally, which was imprecise. Recorded as a reminder that the "read the source, don't guess" rule applies to root causes as much as to behaviour.

Confirmed experimentally — 29 September. Two automated runs differing only in the nonce source (`hcuWEvxGbvRgt7e`, SDK-shaped; `Q9MaDZyfpJhjiDp`, service-generated): this check and F-02's move from WARNING/FAILURE to SUCCESS, and nothing else changes. See the Day 4 entry.

Impact. The nonce exists to bind a wallet's presentation to a specific request and prevent replay. A value derived from the current clock is predictable to within milliseconds, substantially weakening that guarantee. In a national identity deployment this is a meaningful security property, not a cosmetic issue.

ON HOLD — 2 October. The Inji 1.0 line already generates the nonce with `crypto.getRandomValues`. The issue below must not be filed as drafted; whether it is worth raising for the 0.18.x line is a question for the mentors.

Proposed action (superseded). File upstream against `mosip/inji-verify` as a single issue covering F-01 and F-02 together, since they share one root cause and one fix. Recommend `crypto.getRandomValues` over 32 bytes, base64url-encoded without padding. Draft prepared at `docs/upstream/ISSUE-01-nonce-entropy.md`; not yet filed.

F-02 — Nonce contains non-URL-safe characters MEDIUM CONFIRMED

Component: Inji Verify 0.18.2 **Spec:** OID4VP 1.0 Final, §5.2

Evidence.

> `CheckForInvalidCharsInNonce` — FAILURE
> "Non URL safe characters found in nonce. This may introduce interoperability issues."
> `invalid_chars: ["="]`

Observation. The nonce uses standard base64 with `=` padding rather than base64url. The `=` character is not URL-safe and requires percent-encoding when the nonce travels in a query string, which not all wallets handle consistently.

Root cause. Identical to F-01 — `btoa` in `inji-verify-sdk/src/utils/api.ts:11` emits standard base64 with `=` padding rather than base64url.

Impact. Interoperability risk with third-party wallets. Same root cause as F-01 and fixed by the same change; the two are filed upstream as one issue.

F-03 — No DCQL support; test plan cannot proceed HIGH CONFIRMED

Component: Inji Verify 0.18.2 **Spec:** OID4VP 1.0 Final, §6

Evidence — conformance suite.

> `ExtractDCQLQueryFromAuthorizationRequest` — FAILURE
> "dcql_query not found in authorization request parameters"

Evidence — source. A case-insensitive search for `dcql` across `verify-service/src/main` returns zero matches. Credential selection is implemented entirely via `presentation_definition` with `input_descriptors` (DIF Presentation Exchange), set in `VerifiablePresentationRequestServiceImpl` as the `presentation_definition_uri` claim.

Impact. This terminates the test. Because every module in the 1.0 Final verifier plan requires a DCQL query, **Inji Verify 0.18.2 cannot pass any test in that plan.** This is the single most consequential finding of Day 1 and is not fixable by configuration.

Open question for mentors. See Q1.

F-04 — Uses superseded `presentation_definition` MEDIUM CONFIRMED

Component: Inji Verify 0.18.2 **Spec:** OID4VP 1.0 Final

Evidence.

> `CheckNoPresentationDefinitionInVpAuthorizationRequest` — WARNING
> "Authorization request contains legacy 'presentation_definition'. VP1 Final uses 'dcql_query' for credential selection, so sending both is unnecessary and may indicate draft and final request formats are being mixed."

Relationship to F-03. Two faces of the same gap: Verify implements the earlier draft's query model. Recorded separately because a maintainer may address them independently (e.g. support both during a transition period).

F-05 — `client_metadata` absent in inline mode LOW RESOLVED — NOT A DEFECT

Resolved 2 October: our artefact. The 0.18.x SDK adds `client_metadata` (`client_name` and `vp_formats`) to every inline request when it builds the request URI (`OpenID4VPVerification.tsx`, lines 124–130). Our `build-oid4vp-uri.py`, and the runner copied from it, did not. The "client_metadata not present" warning in the Day 1 run and in run `hcuWEvxGbvRgt7e` was produced by our tooling, and both runs share the artefact — which is why they matched each other. The runner now reproduces the SDK. One caveat stands: the 0.18 SDK uses the pre-Final key `vp_formats`, while 1.0 Final requires `vp_formats_supported`, so on 0.18.2 that check would likely still fail, for a different reason. On 1.0 the SDK uses the Final key.

Original entry follows.

Component: Inji Verify 0.18.2 **Spec:** OID4VP 1.0 Final, §5.1; Annex B.2.2, B.3.4

Evidence.

`VP1FinalValidateVpFormatsSupportedInClientMetadata` — FAILURE
"client_metadata is missing or not a JSON object"

`VP1FinalCheckForUnexpectedParametersInVpClientMetadata` — WARNING
"client_metadata not present in request but it is mandatory to include vp_formats in it"

Why this is not yet confirmed. In the `did` (by-reference) path, Verify's signed request object *did* contain `client_metadata` with a full `vp_formats` block advertising `ldp_vp` and `vc+sd-jwt`. In the `pre_registered` (inline) path, the `/vp-session-request` API response contained no `client_metadata` field, so the URI we constructed could not include one. It is not yet established whether Verify omits it in inline mode, or whether the UI adds it when building the QR code and our reconstruction simply dropped it.

Action required before reporting. Inspect the QR payload the Verify UI actually produces in `pre_registered` mode and compare against the API response.

F-06 — No `direct_post.jwt`; HAIP certification impossible HIGH CONFIRMED

Component: Inji Verify 0.18.2 **Spec:** OID4VP 1.0 Final / HAIP

Evidence — source.

```
verify-service/src/main/java/io/inji/verify/shared/Constants.java
public static final String RESPONSE_MODE = "direct_post";
```

This is a `static final` constant, not a configuration property or environment variable. No code path produces the encrypted `direct_post.jwt` variant.

Evidence — conformance suite. The HAIP verifier plan (*OpenID for Verifiable Presentations 1.0 Final/HAIP: Test a verifier*) offers only `direct_post.jwt` in its Response Mode variant.

Impact. Inji Verify cannot be submitted for HAIP self-certification in its current state. HAIP is the profile relevant to high-assurance national identity deployments, which is MOSIP's own domain.

Open question for mentors. See Q2.

F-07 — DID document verification method identifier inconsistency MEDIUM CONFIRMED

Component: Inji Verify 0.18.2 **Spec:** W3C DID Core

Evidence. Retrieved from `https://<host>/v1/verify/did.json`:

`"id": "did:web:<host>:v1:verify"` (document subject)
`"verificationMethod[0].id": "did:web:<host>:verify#key-0"` (no :v1:)
`"assertionMethod[0]": "did:web:<host>:verify#key-0"` (no :v1:)

Origin. The default Docker Compose environment variables differ: `INJI_DID_VERIFY_URI` includes the `v1` path segment while `INJI_DID_VERIFY_PUBLIC_KEY_URI` does not.

Impact. Under `did:web` resolution rules, the key's identifier resolves to `https://<host>/verify/did.json`, a path that does not exist on the deployment. A strict relying party resolving the verification method by its own identifier would fail.

F-08 — No mdoc / ISO 18013-5 support LOW CONFIRMED

Component: Inji Verify 0.18.2

Evidence. Searches for `mso_mdoc`, `iso_mdl` and `mdoc` across `verify-service/src/main` return zero matches.

Impact. The `iso_mdl` credential format variant of the verifier plan cannot be exercised. Recorded for completeness; likely a deliberate scope decision rather than a defect.

F-09 — Client identifier prefix has no overlap with 1.0 Final MEDIUM CONFIRMED

Component: Inji Verify 0.18.2 **Spec:** OID4VP 1.0 Final, §5.9.2

Evidence.

```
EnsureClientIdMatchesResponseUri — FAILURE
"For redirect_uri client_id prefix, the client_id (without prefix) must equal response_uri"
client_id: "inji-verify-ui"  response_uri: "https://<host>/v1/verify/vp-submission/direct-post"
```

Observation. Inji Verify supports two client identifier schemes, `did` and `pre_registered`. The 1.0 Final verifier plan offers three prefixes: `redirect_uri`, `x509_san_dns`, `x509_hash`. Verify has no X.509 material configured, so the two X.509 options are unreachable, and `redirect_uri` requires a client identifier Verify does not produce. **No variant combination in the plan matches Verify's client identification model.**

Caveat. This failure is partly a consequence of the variant we selected. The reportable observation is the absence of overlap, not that any single value is "wrong".

Open question for mentors. See Q3.

F-10 — Docker Compose database script out of date; every VP submission fails with HTTP 500 HIGH CONFIRMED

Component: Inji Verify 1.0.0-alpha.1, and `release-1.0.x` head (6a6ab3b, 1 October) **Kind:** packaging — the service is correct

Observation. Brought up the documented way, with Docker Compose, Inji Verify 1.0 answers every presentation submission (`POST /v2/vp-submission/direct-post`) with HTTP 500.

Evidence — service log (run `gIA512EeJWehcHt`):

```
PSQLException: ERROR: column v1_0.response_code does not exist
at ...VerifiablePresentationRequestServiceImpl.getCurrentRequestStatus(...java:126)
```

Root cause. The Compose stack initialises its database from `docker-compose/db-init/init.sql`, which creates `verify.vp_submission` with five columns. The canonical schema, `db_scripts/inji_verify/ddl/verify-vp_submission.sql`, has eight: it adds `response_code`, `response_code_expiry_at` and `response_code_used`, makes `vp_token` and `presentation_submission` nullable, and adds a primary key. The 1.0 service is written against the canonical schema.

Ruled out first: a stale database from the earlier 0.18.2 stack. The data volume was created at 06:34 on 2 October; the Postgres log shows `init.sql` running and creating four tables; the live table matches `init.sql` column for column.

Confirmed fix. With the canonical `vp_submission` definition substituted into `init.sql`, the 500 disappears and Verify processes submissions normally (runs `tpKVz6upccxpO5U` onward).

Also inferred, not observed: `presentation_submission NOT NULL` would likely break storing a DCQL response, which carries no presentation submission. Not separately tested.

Impact. Anyone evaluating Inji Verify 1.0 through its quick start sees verification fail outright, for a reason that is not in the product.

Proposed action. Report upstream with the fix: regenerate `init.sql` from `db_scripts`. Present on the branch head, so this is the one to file. Draft in `docs/upstream/ISSUE-02-compose-db-init.md`.

F-11 — Shipped DCQL sample uses the superseded format identifier LOW CONFIRMED

Component: Inji Verify 1.0.0-alpha.1 `docker-compose/config/config.json` **Kind:** sample configuration

Evidence. `ValidateDCQLQuery`: *"does not have a value in the enumeration [dc+sd-jwt, mso_mdoc]"* at `$.credentials[0].format`; the shipped *Mock Identity* entry requests `vc+sd-jwt`. OID4VP 1.0 Final, §6.

Configuration, not implementation. Run `ZQqnbYfw46wpmyI`, requesting `dc+sd-jwt`, cleared both DCQL failures and Verify accepted the presentation.

Not to be reported. The `release-1.0.x` head already adds a `dc+sd-jwt` entry to the sample configuration; upstream is addressing it.

F-12 — Certify's authorisation-server metadata cannot carry authorization_endpoint MEDIUM CONFIRMED

Component: Inji Certify `release-1.0.x` (QA image, digest `2c72a78c`) **Kind:** implementation **Spec:** RFC 8414 §2

Observation. With Certify as its own authorisation server — what the pre-authorized grant needs — its metadata advertises `grant_types_supported` including `authorization_code` and `response_types_supported: [code]`, but no `authorization_endpoint`; it offers `interactive_authorization_endpoint` instead. RFC 8414 §2 requires `authorization_endpoint` unless no supported grant uses it.

Implementation, not configuration. `OAuthAuthorizationServerMetadataDTO` has no such field, so no setting can add it.

Consequence. The OpenID Foundation issuer plan's FAPI 2.0 base requires the field for every module (`CheckServerConfiguration`), so no module gets past its first check (run `LY-hOSkqpZXhz7jP`). Every later Certify result was obtained through a test shim in our gateway that injects the field; the pre-authorized flow never calls it.

Two readings. Certify follows a newer OID4VCI draft's interactive authorisation, so it may consider the classic endpoint out of scope; but then it should not advertise `authorization_code` with `response_types_supported: [code]`. And the suite could arguably relax its check for pre-authorized-only tests. Both are worth raising.

F-13 — Credential request rejects unrecognised parameters MEDIUM CONFIRMED

Component: Inji Certify `release-1.0.x` **Kind:** implementation **Spec:** OID4VCI 1.0 Final §8.2 — "The Credential Issuer MUST ignore any unrecognized parameters."

Evidence. The happy-flow module adds a random top-level field to the credential request. Certify answers 400 `invalid_request`; its log: `UnrecognizedPropertyException: Unrecognized field "W38QWFYMxaOFEhkw" (class io.mosip.certify.core.dto.CredentialRequest), not marked as ignorable (2 known properties: "credential_configuration_id", "proofs")` (run `C5ccbcnbc9sRWMk`). The same rejection hits the spec-defined `credential_identifier`.

Impact. Any wallet that sends a newer or extension parameter cannot obtain a credential. Blocks the issuer happy flow.

Proposed fix. Ignore unknown properties when reading `CredentialRequest` (e.g. `@JsonIgnoreProperties(ignoreUnknown = true)`). Present on the branch head: worth filing.

F-14 — Wrong error code for an unknown credential configuration LOW CONFIRMED

Component: Inji Certify `release-1.0.x` **Spec:** OID4VCI 1.0 Final §8.3.1.2

Evidence. For an unknown `credential_configuration_id` Certify answers `invalid_credential_request`; the spec defines `unknown_credential_configuration` (module `fail-unknown-credential-configuration`, plan `89gcoUu8shPu1`).

F-15 — Authenticated wallets' proofs rejected in the pre-authorized flow MEDIUM CONFIRMED (behaviour)

Component: Inji Certify `release-1.0.x` **Spec:** OID4VCI 1.0 Final, App. F.1

Evidence. The suite authenticates to the token endpoint with `private_key_jwt` as `oidf-conformance-wallet`. Certify issues the access token with an *empty* `client_id`, then rejects the wallet's valid proof: `JWT iss claim has value oidf-conformance-wallet, must be <empty>`. Four modules stop here.

Two readings — the classification is not settled. App. F.1 requires the proof's `iss` to be omitted only when the token came from *anonymous* pre-authorized access. (a) A defect: an authenticated client should have its `client_id` bound into the token, and its proof accepted. (b) A design limit: Certify supports only anonymous pre-authorized access — in which case it should refuse or not advertise client authentication rather than silently ignore it. Either way, a wallet that authenticates cannot obtain a credential. A question for the maintainers before any report.

F-16 — Revoked SD-JWT credentials are accepted: the status check silently does nothing HIGH CONFIRMED

Component: Inji Verify 1.0.0-alpha.1 and `release-1.0.x`, through MOSIP's `vc-verifier` (1.9.0 and `develop`) **Spec:** SD-JWT VC s3.4; Token Status List s8.3

Evidence. Suite build of 27 October, module `present-revoked-credential` (`logs/2026-10-27/...-r1i7jlHHrzwZDTc.json`): the credential carries a status-list reference marking it revoked. `EnsureVerifierFetchedRevocationList`: *the verifier did not fetch the revocation list.* Verify's own verdict, fetched by the harness: `allChecksSuccessful=true` — the revoked credential is reported valid. Every other SD-JWT module now carries the same warning.

Root cause, in the source. Verify asks for the `revoked` status of each credential (`VerifiablePresentationSubmissionServiceImpl.verifyAndGetCredentialStatus`). `vc-verifier` implements status checking only for LDP credentials (`LdpStatusChecker`); the SD-JWT type inherits the default, which throws `UnsupportedOperationException`. `CredentialsVerifier.getCredentialStatus` catches that, logs a warning and returns an *empty* status map; Verify's `Utils.checkIfVCIsRevoked` reads an empty map as not revoked. The check fails open, and nothing in the verdict says it was skipped.

Classification. An implementation gap (status lists for SD-JWT are not implemented), made worse by the fail-open handling: a verifier that cannot check revocation should say so in its verdict rather than report the credential valid. The fix belongs in `vc-verifier` (a Token Status List checker for SD-JWT), with Verify surfacing an unchecked status. In the gate this module is baselined FAILED and acknowledged as F-16.

Observations on Inji Certify — not findings

- **Token not DPoP-bound.** The pre-authorized token came back as `token_type: Bearer` with no `cnf`, although the wallet sent a DPoP proof. The head contains DPoP code; whether it applies to this grant is unverified.
- **Unauthenticated configuration API.** `POST /credential-configurations` accepted a new credential with no credentials, on an instance reachable through the public tunnel. Possibly by design for this local profile; unverified for a production profile.
- **No signed issuer metadata** — an optional OID4VCI feature.
- The TLS cipher warning in one module concerns ngrok's TLS termination, not Certify.

4 Open Questions for Mentors

These are the questions we would like to put to MOSIP mentors. Each is phrased to be answerable without our setup in front of them.

Note on channel. The kickoff seminar and mentor AMA scheduled for the start of the event did not take place, so the community forum is currently the only route to an answer, and no answer is guaranteed. Where a question shapes the build — Q1 above all — our approach is to make the affected choice a configuration value and to state the assumption plainly in the submission, rather than to block on a reply.

Q1 — Target specification version.
Inji Verify 0.18.2 implements OID4VP credential selection via `presentation_definition` (Presentation Exchange), whereas OID4VP 1.0 Final requires `dcql_query`. Consequently no test in the 1.0 Final verifier plan can complete. Which specification version should our harness target as the benchmark: 1.0 Final (accepting that the current baseline is a full set of failures), or the ID2 draft that the shipped implementation appears to target? Is DCQL support on the Inji roadmap, and if so for which release?

Q2 — HAIP certification intent.
The problem statement asks us to document "how to submit for self-certification". The HAIP verifier plan mandates `direct_post.jwt`, but `RESPONSE_MODE` is a `static final` constant set to `direct_post` in `Constants.java`. Is encrypted response mode planned? Should our harness include the HAIP plan as a known-failing suite to track progress toward certification, or exclude it until the capability exists?

Q3 — Client identifier scheme.
Verify supports `did` and `pre_registered` client identifier schemes; the 1.0 Final plan offers `redirect_uri`, `x509_san_dns` and `x509_hash`. Is there a supported configuration that allows an Inji Verify deployment to present a client identifier the 1.0 Final conformance plan can validate, or is this a known gap?

Q4 — Benchmark semantics for `api-testrig`.
The problem statement asks for results "gated against a conformance benchmark". Given that the honest current baseline is "all modules fail at DCQL extraction", should the benchmark be (a) a frozen snapshot of today's pass/fail set, so the gate catches regressions only, or (b) an aspirational target that the build fails against until conformance improves? Our current design assumes (a), with an expected-failures file under version control.

Q5 — Sample configuration shipped with Inji Verify.
The default `config.json` ships five credential types, four of which use `ldp_vc` with `clientIdScheme: did`. That combination cannot be exercised by the 1.0 Final plan at all. Is the shipped sample configuration intended to be certification-representative, or purely a demonstration fixture? This affects whether we report the configuration itself as a finding.

Q6 — Interactive tests and automation.
Every module in the verifier plan is interactive: it waits for a human to trigger a flow in the verifier and paste the resulting authorisation request, and the happy-flow module also requires a screenshot upload before it reaches REVIEW status. Our harness can drive request generation via Verify's own API, but the screenshot-upload requirement appears to need `POST /api/log/{id}/images`. Is a fully unattended run of the verifier plan considered achievable, or is a documented semi-automated flow the expected deliverable?

Substantially answered by our own investigation on Day 3, and retained here only for confirmation. The authorisation request is delivered by an HTTP GET to the test's exposed `authorization_endpoint`, and the screenshot placeholder is filled by the suite's own `browser`

automation against an HTML stand-in page it serves itself. Both are source-derived and await confirmation against a live run.

Q7 — Scope of Inji Certify work.
The `docker-compose-injistack` setup requires a PKCS12 keystore obtained through Mimoto onboarding, a publicly resolvable DID endpoint, and an external authorisation server. For a two-person team on a four-week timeline, is there a lighter-weight Certify configuration suitable for conformance testing, or is the Collab sandbox the intended target for the issuer plan?

Q8 — Presentation definition id (raised 29 September).
`verify-ui` sends every presentation definition with the same hard-coded id (`vpVerificationState.ts:59`), while the definition's contents vary with the credentials selected. Is anything on the Inji side — or in Inji Wallet — expected to rely on that id distinguishing definitions? We have not treated this as a defect, because we have found no consequence of it.

5 Reproduction

To reproduce the Day 1 baseline run:

1. Follow `docs/SETUP.md` to install prerequisites and clone the repositories.
2. Follow `docs/RUNBOOK.md` to start the conformance suite, Inji Verify, and ngrok.
3. In `inji-verify/docker-compose/config/config.json`, ensure the *Mock Identity (SD JWT)* entry has `"clientIdScheme":"pre_registered"`.
4. In the conformance suite, create the plan *OpenID for Verifiable Presentations 1.0 Final: Test a verifier — alpha tests* with the variant recorded in Section 2.
5. Run module 01, `oid4vp-1final-verifier-happy-flow`.
6. In the Verify UI, request the *Mock Identity (SD JWT)* credential; capture the `/v1/verify/vp-session-request` response.
7. Run `python3 scripts/build-oid4vp-uri.py` on that response and paste the resulting URI into the suite. The request expires 300 seconds after issue.
8. Export the signed log and store it under `logs/<date>/`.

Since Day 4 the whole reproduction above is one command: `./run-conformance.sh -component verify`. *Findings from modules 02–11 and the ID2 verifier plan to be appended as they are executed.*